



Università degli Studi di Salerno

Corso di Software Architecture Design

Relazione Tecnico-Metodologica

Music Playlist Manager

Gruppo 01

0622702428 – Domenico Nappo

IE22700035 – Arianna Paletta

IE22700189 – Vittorio Postiglione

IE22700199 – Mattia Sanzari

IE22700027 – Debora Villano

Anno Accademico 2025/2026

Indice

1	Environment Configuration	1
2	Software Architecture: The Model-View-Controller (MVC) Pattern	1
3	Mockup and User Interface	4
4	Coding Convention	5
4.1	Naming Convention	5
4.2	Code Commenting Policy	6
4.3	Formal Documentation (Javadoc)	6

1 Environment Configuration

Per garantire la corretta collaborazione tra i membri del team e la gestione del ciclo di vita del software, è stato configurato il seguente ambiente di lavoro:

- **Version Control (GitHub):** https://github.com/CupoMeridio/Java_Music_Playlist_Manager.git
- **Project Management (Trello):** <https://trello.com/b/181884u5>
- **Mockup (Miro):** https://miro.com/app/board/uXjVHPUrlcg=/?share_link_id=497695099650
- **Ambiente di Sviluppo (IDE):** IntelliJ IDEA / NetBeans
- **Linguaggio e SDK:** Java 21 (JDK 21)
- **Build and Dependency Management:** Apache Maven
- **GUI Framework:** JavaFX
- **Testing Framework:** JUnit 5

2 Software Architecture: The Model-View-Controller (MVC) Pattern

Per lo sviluppo dell'applicazione *Music Playlist Manager*, il team ha scelto di adottare l'architettura **Model-View-Controller (MVC)**. Questa scelta è stata dettata dalla necessità di separare nettamente la logica di business dell'applicazione dalla sua interfaccia grafica, garantendo al contempo flessibilità e manutenibilità del codice nel tempo.

L'architettura è stata strutturata secondo le tre componenti fondamentali:

- **Model:** Gestisce i dati del sistema e le operazioni associate, incapsulando interamente lo stato dell'applicazione. Nel caso specifico di questo progetto, definisce i modelli dei brani, delle playlist e della libreria.
- **View:** Definisce e gestisce il modo in cui i dati sono presentati all'utente. All'interno dell'applicazione, si occupa di visualizzare le informazioni fornite dal Model, come la lista dei brani, le playlist attuali e la barra di avanzamento.
- **Controller:** Funge da intermediario tra il Model e la View, gestendo gli input dell'utente e aggiornando lo stato dell'applicazione.

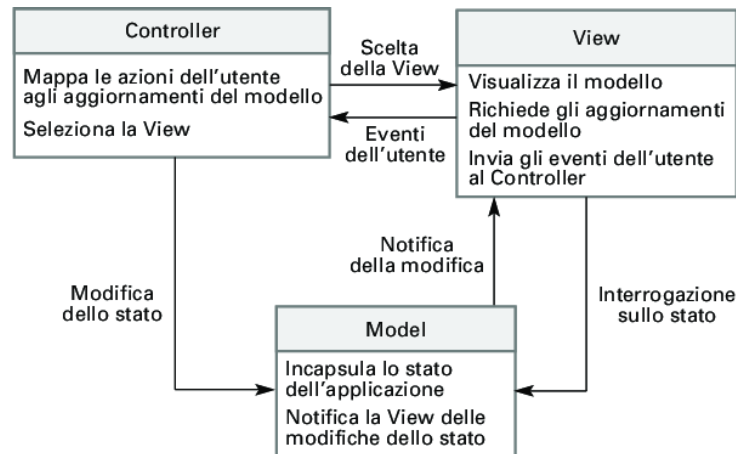


Figura 1: Diagramma delle interazioni tra le componenti MVC.

L'adozione di questo pattern risponde direttamente ai criteri di valutazione richiesti dal progetto, in particolare:

- **Rappresentazione Multipla e Indipendente:** Permette di separare i dati core dall'interfaccia grafica. Di conseguenza, gli stessi dati possono essere mostrati contemporaneamente in modi differenti (ad esempio, il brano in esecuzione può essere visualizzato sia nella lista principale sia nel player di controllo) senza che il Model debba conoscere i dettagli della View. Qualsiasi modifica ai dati si riflette automaticamente su tutte le rappresentazioni attive.
- **Adattabilità a Evoluzioni Future:** Risulta una scelta ideale quando non sono noti a priori i requisiti futuri di interazione e presentazione dei dati.
- **Alta Coesione e Basso Accoppiamento:** La separazione dei compiti permette a ogni pacchetto di fornire servizi altamente correlati, riducendo le dipendenze non necessarie tra i moduli.
- **Gestione del Real-Time:** L'MVC è ideale per gli aggiornamenti dinamici. Durante il playback, il progresso gestito dal Model si riflette in tempo reale sulla View (es. barra di avanzamento e cronometri), garantendo una reattività ottimale.
- **Facilità di Testing:** Separando la logica di gestione dei brani dall'interfaccia grafica, è possibile implementare test unitari (*JUnit*) più efficaci sulla logica di business, coprendo adeguatamente le classi critiche.
- **Supporto a Funzionalità Avanzate:** Strutturare il progetto in MVC rende più agevole l'implementazione del sistema di **Undo** e del sistema di **Smart Playlists**, poiché la logica di filtraggio può essere sviluppata e testata isolatamente all'interno del Model.

Il pattern adottato però porta con se anche un piccolo svantaggio:

- **Sovraccarico e Complessità:** L'architettura potrebbe richiedere altro codice o un codice complesso quando il modello dei dati e le interazioni richieste dall'applicazione sono in realtà semplici.

3 Mockup and User Interface

Al fine di definire l'esperienza utente (UX) e l'interfaccia (UI) prima della fase di codifica, sono stati realizzati i mockup delle schermate principali dell'applicazione. Questi prototipi statici hanno permesso al team di visualizzare la disposizione logica degli elementi (come la libreria musicale, i controlli di riproduzione e i pannelli di gestione) e di validare i flussi di navigazione fondamentali.

Si specifica, tuttavia, che tali mockup sono da intendersi esclusivamente come linee guida progettuali e punti di riferimento per lo sviluppo; l'interfaccia grafica (GUI) finale, implementata tramite JavaFX, potrebbe pertanto differire parzialmente nei dettagli visivi, nel layout o nel posizionamento dei componenti a seguito di ottimizzazioni emerse in fase di codifica.

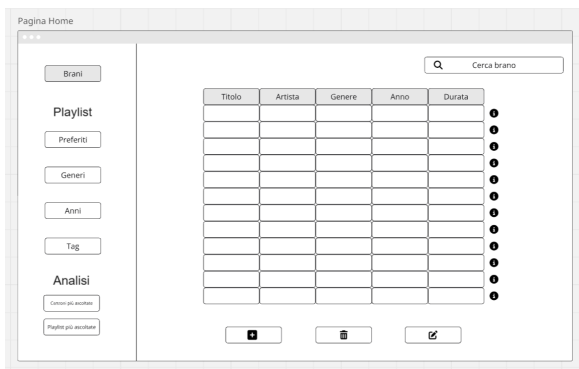


Figura 2: Schermata principale.

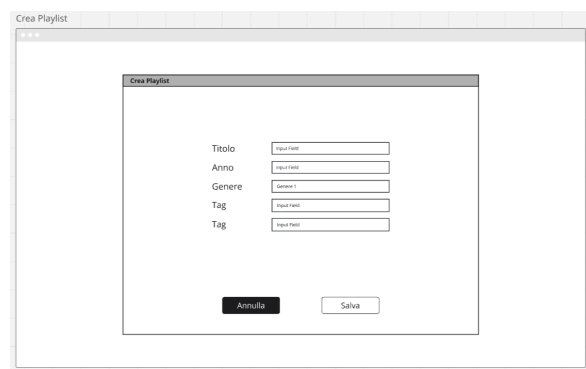


Figura 3: Gestione e creazione Playlist.

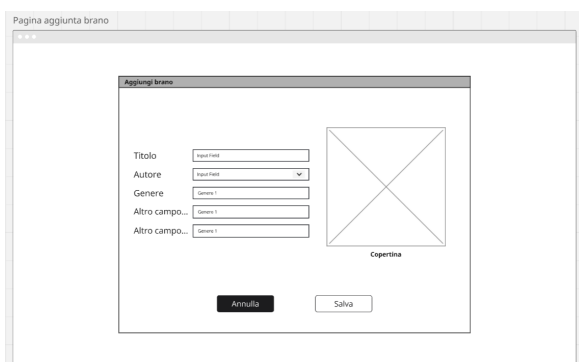


Figura 4: Gestione e aggiunta Track.

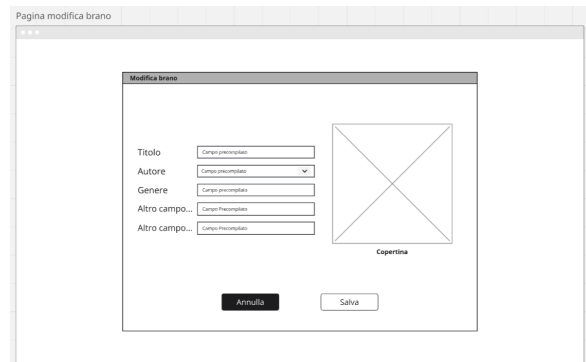


Figura 5: Schermata di modifica di una track.

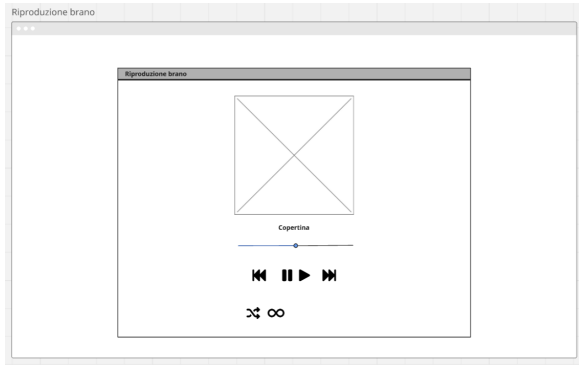


Figura 6: Schermata di riproduzione track.

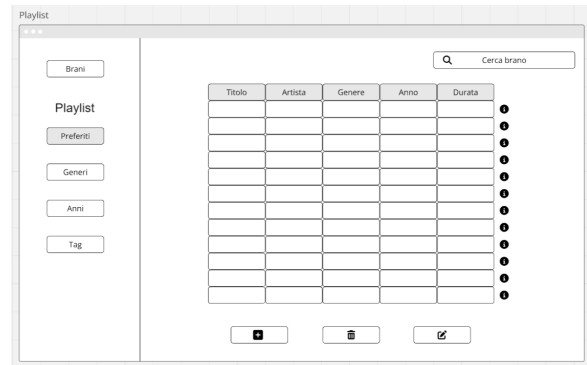


Figura 7: Schermata vista generale di una playlist.

4 Coding Convention

In linea con le metodologie di sviluppo **Agile** e i **principi di buon design del software**, il team ha adottato un set di convenzioni di codifica rigorose. L'obiettivo è minimizzare il debito tecnico e garantire che il codice sia facilmente leggibile, manutenibile e testabile da ogni membro del gruppo.

Le principali convenzioni adottate includono:

4.1 Naming Convention

Il team segue le convenzioni standard del linguaggio Java per garantire l'uniformità del codice:

- **Classi e Interfacce:** Dichiarate in *UpperCamelCase* (es. `PlaylistManager`, `Audio Player Simulato`), utilizzando sostantivi chiari e autoesplicativi.
- **Metodi e Variabili:** Dichiarati in *lowerCamelCase* (es. `playTrack()`, `currentPosition`). I metodi devono iniziare con un verbo che ne esprima chiaramente l'azione.
- **Costanti:** Dichiarate interamente in maiuscolo con *Snake_Case* (es. `MAX_VOLUME`, `DEFAULT_GENRE`) per evitare l'utilizzo di "magic numbers" o valori hard-coded all'interno della logica di business.

Al fine di mantenere una coerenza semantica ed evitare ambiguità terminologiche tra i membri del gruppo, è stato definito un **Glossario dei Termini del Dominio**. Ad esempio, si è concordato l'uso del termine **Track** per identificare il singolo brano musicale (coerentemente con le richieste delle User Epics) e del termine **Playback** per fare riferimento allo stato e alle modalità di riproduzione simulata.

4.2 Code Commenting Policy

I commenti non devono essere ridondanti o spiegare *cosa* fa una singola riga di codice evidente, bensì il *perché* di una determinata scelta implementativa complessa o non banale.

Il team aderisce attivamente alla "**Regola del Boy Scout**": *"Lascia il campo base più pulito di come lo hai trovato"*. Ogni qualvolta un membro del team lavora su una classe per implementare un task, è tenuto a rifattorizzare eventuali frammenti di codice obsoleti, eliminare avvisi del compilatore e migliorare la chiarezza dei commenti preesistenti.

4.3 Formal Documentation (Javadoc)

Per garantire la manutenibilità del software a lungo termine, ogni classe, interface e metodo pubblico o protetto deve essere preceduto da un blocco di documentazione formale in formato **Javadoc**. Questo approccio consente la generazione automatica della documentazione tecnica del progetto.

I metodi devono obbligatoriamente specificare il loro contratto comportamentale attraverso l'uso dei tag standard:

- **@param**: Descrive il significato, il tipo e i vincoli di validità di ciascun parametro di ingresso.
- **@return**: Specifica il valore restituito dal metodo e il suo significato nel contesto applicativo.
- **@throws** (o **@exception**): Indica le eccezioni controllate o non controllate che il metodo può sollevare, descrivendo le condizioni che causano tale comportamento anomalo.

Di seguito viene riportato uno standard di riferimento adottato dal team:

```
1 /**
2  * Modifica la traccia correntemente in riproduzione nel
3   * simulatore.
4  * Se il player e' in esecuzione, la riproduzione viene
5   * interrotta
6  * e avviata immediatamente con il nuovo brano.
7  *
8  * @param track Il nuovo oggetto Track da riprodurre. Non puo'
9   * essere null.
10  * @return true se il cambio di traccia e' avvenuto con successo,
11   * false altrimenti.
12  * @throws IllegalArgumentException se il parametro track e'
13   * nullo.
14  */
```

```
10 public boolean changeCurrentTrack(Track track) {  
11     if (track == null) {  
12         throw new IllegalArgumentException("La traccia non puo'  
            essere nulla.");  
13     }  
14  
15     // Logica di business...  
16     return true;  
17 }
```

Listing 1: Esempio di documentazione Javadoc applicata al dominio